

Homework Week 6 [45 points +5 extra points]

Due: Oct 7, 2025 11:59pm PT.

Upload a zip archive of your project on <https://brightspace.usc.edu> .

Your project must consist of a folder `hw6_<first_name>_<last_name>/` and python files (python modules) and data file(s) inside:

- `hw6_<first_name>_<last_name>/vehicle.py` # place your functions from task 1 here
- `hw6_<first_name>_<last_name>/library.py` # place your functions from tasks 2 here
- `hw6_<first_name>_<last_name>/product_inventory.py` # place your functions from tasks 3 here
- `hw6_<first_name>_<last_name>/shopper.py` # place your functions from tasks 4 here
- `hw6_<first_name>_<last_name>/tests.py` # place your tests here, see example in HW4

Task 1. Implement a Vehicle Class. [10 points + 5 extra]

Create a class to model basic vehicle properties and behaviors.

Attributes:

- `vehicle_id`: str
- `make`: str
- `model`: str
- `year`: int

Methods:

- `__init__(self, vehicle_id, make, model, year):`
 - - Take four arguments `vehicle_id`, `make`, `model` and `year`, and store them in attributes
- `__str__(self):`
 - - You need to define a custom `__str__` method to properly print its content
 - - The format is shown below, where angle brackets are to be replaced with actual values:

```
Vehicle(vehicle_id='<vehicle_id>', make='<make>', model='<model>', year=<year>)
```
- `__repr__(self):`
 - - Return the same content as `__str__`

Example Usage:

DSCI 510 – Fall 2025 – University of Southern California

Instructor: Dr. Alexey Tregubov

Input:

```
car = Vehicle("VIN123", "Toyota", "Camry", 2020)
print(car)
```

Output:

```
Vehicle(vehicle_id='VIN123', make='Toyota', model='Camry', year=2020)
```

For extra credit (+5 points): make a program with command line arguments from your ***vehicle.py***. It is similar to your homework 5 task4. Your program should have 4 command line arguments: `vehicle_id`, `make`, `model`, `year`. Your program should create an object of class `Vehicle` and print it (see input/output example above).

Update: Make sure that year is in the range [1900, 2025]

Task 2. Implement a Library Member Class. [10 points]

Develop a `LibraryMember` class to represent library members.

Attributes:

- `member_id`: str
- `name`: str
- `borrowed_books`: list containing book titles

Methods:

- `__init__(self, member_id, name)`:
 - Take two arguments: `member_id`, and `name`, and store them in attributes
 - `borrowed_books` should be initialized to an empty list.
- `add_book(self, book_title)`:
 - Update `borrowed_books` when a book is borrowed.
 - Update `borrowed_books` when a book is borrowed.
- `remove_book(self, book_title)`:
 - Update `borrowed_books` when a book is borrowed.
- `count_books(self)`:
 - Total number of `borrowed_books`
- `__str__(self)`:
 - Return a string showing the member's ID, name and the total number of books borrowed.
- `__repr__(self)`:
 - Return the same string as `__str__`.

Example Usage:

Input:

DSCI 510 – Fall 2025 – University of Southern California

Instructor: Dr. Alexey Tregubov

```
member = LibraryMember("001", "Alice")
member.add_book("The Great Gatsby")
print(member)
```

Output:

```
LibraryMember(member_id='001', name='Alice', total_borrowed=1)
```

Update: 1. You are not allowed to add the same book multiple times. Raise an error

Task 3. Implement a Product Class for an Inventory System. [10 points]

Create a Product class for managing inventory in a retail system.

Attributes:

- product_code: str
- name: str
- price: float
- quantity_available: int

Methods:

- `__init__(self, product_code, name, price, quantity_available)`: Initialize the attributes.
 - Take four arguments: product_code, price, name, and quantity_available and store as attributes
- `update_price(self, new_price)`:
 - Update the product's price.
- `add_stock(self, quantity)`:
 - Add to quantity_available.
- `sell_product(self, quantity)`:
 - Subtract from quantity_available when a sale is made. If quantity_available is less than sell quantity, do not perform any sale
- `__str__(self)`:
 - Return a string with the product's details
- `__repr__(self)`:
 - Return the same string as `__str__`.

Example Usage:

Input:

```
product = Product("12345", "Coffee Mug", 12.99, 100)
product.sell_product(2)
print(product)
```

Output:

DSCI 510 – Fall 2025 – University of Southern California

Instructor: Dr. Alexey Tregubov

```
Product(product_code='12345', name='Coffee Mug', price=12.99,  
quantity_available=98)
```

Task 4. Implement a Shopper class and its subclass. [15 points]

Implement the Shopper class with these attributes:

- name (str): The shopper's name.
- shopper_id (str): A unique identifier for the shopper.
- cart: A data structure of your choice to store Product objects and their associated quantities.

Methods to Implement:

1. Constructor:
 - a. Initialize the shopper's name, shopper_id, and cart.
2. add_item(product, quantity):
 - a. product: An instance of the Product class.
 - b. quantity: A non-negative integer.
 - c. Functionality:
 - i. If the product is not already in the cart, add it with the given quantity.
 - ii. If the product is already present, increase its quantity by the specified amount.
3. remove_item(product, quantity):
 - a. product: An instance of the Product class.
 - b. quantity: A non-negative integer.
 - c. Functionality:
 - i. If the product is not in the cart, do nothing.
 - ii. If the product exists and the quantity to remove is equal to or exceeds the current quantity, remove the product from the cart.
 - iii. Otherwise, reduce its quantity by the specified amount.
4. cart_total():
 - a. Functionality:
 - i. Calculate and return the grand total price for all items in the cart as a rounded float.
 - ii. Use the product's current price attribute in the calculation (to account for potential price updates).

DSCI 510 – Fall 2025 – University of Southern California

Instructor: Dr. Alexey Tregubov

Update: If there is not enough product in stock, method **Product:sell_product()** should give you an error (raise an error).

Implement the PremiumShopper (Inheritance).

Implement the PremiumShopper class as a subclass of Shopper.

Inheritance Requirements:

- PremiumShopper should inherit all attributes and methods from the Shopper class.
- Additional Functionality:
 - Discount Policy:
 - Premium shoppers receive a 3.5% discount on their entire cart total if the total is \$50 or more.
 - Method Override:
 - Override the cart_total() method in PremiumShopper to apply the discount when applicable.

Provide multiple usage examples with every method.

Update: If you add something to the cart you need to track it in the inventory. By adding something to the cart you also reduce the stock of the product via **Product:sell_product()**. For the product with insufficient inventory, you cannot complete this transaction. If there is not enough product in stock, method **Product:sell_product()** should give you an error. Therefore, **Shopper:add_item()** should also raise an error.